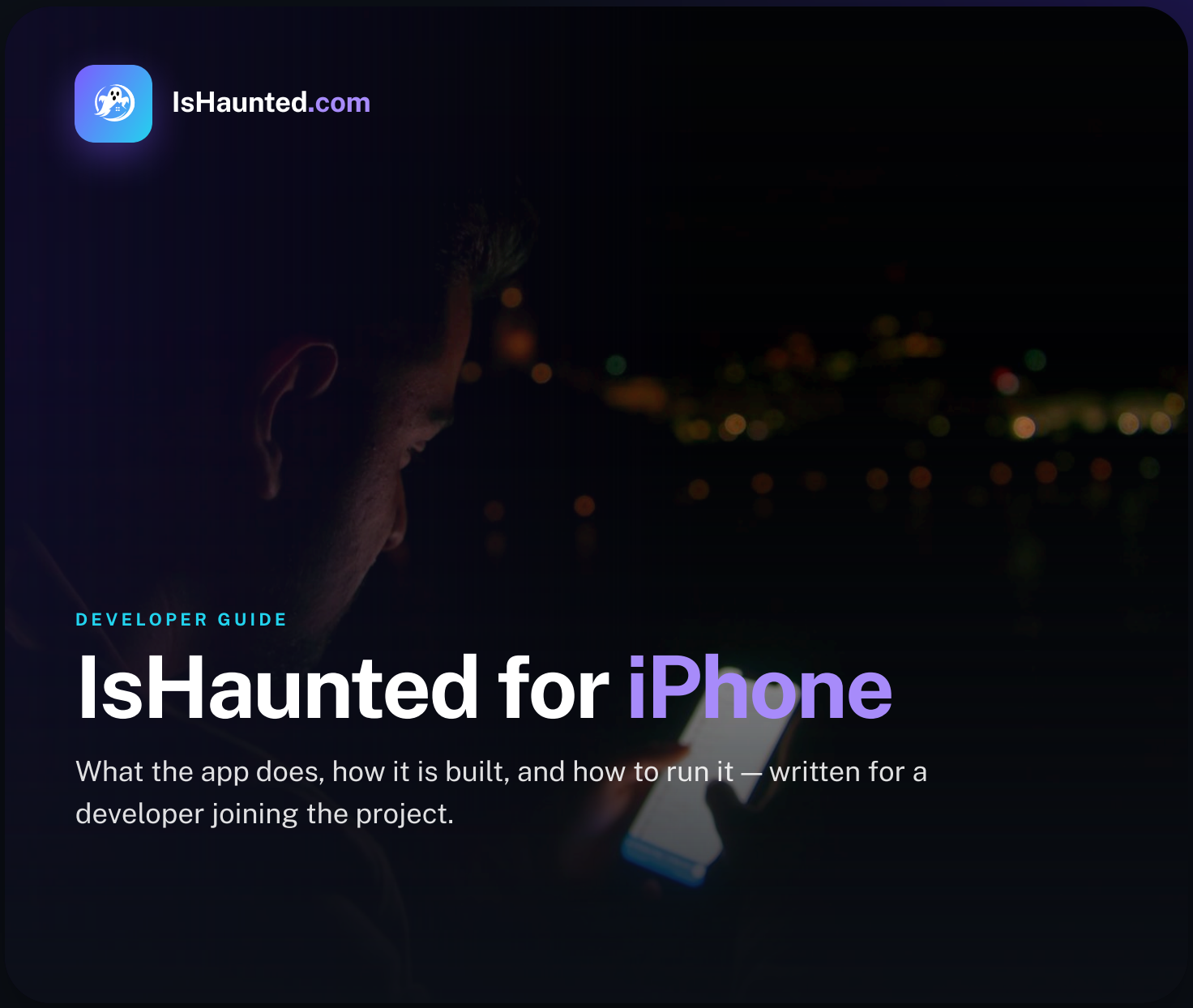
IsHaunted.com



DEVELOPER GUIDE

IsHaunted for iPhone

What the app does, how it is built, and how to run it — written for a developer joining the project.

DEVICE	SCREENS	BUILT
iPhone	36	10/23/2026

Every screenshot in this document shows simulated data. The accounts, cases, feed posts and recordings are seeded; the photographs and audio waveforms are generated, and each is captioned `SIMULATED`. Nothing here is a real investigation or a real person. Screens were captured on an iPhone simulator in dark mode.

What this is

IsHaunted is a platform for paranormal investigation groups and the people who ask them for help. The website runs the business — cases, clients, groups, billing. **This app is the part that goes into the building at 2am.**

It is a native Swift 6 / SwiftUI universal app. One target runs on both devices and picks its shell by size class: a `TabView` across the bottom on iPhone. This document covers the iPhone; the iPad has its own.

> **It is a pure REST client.** The app talks to `Ben.Data.WebApi` and nothing else. `Ben.iOS/` is deliberately not referenced by `Ben.slnx`, so the website and the API never see it and cannot be broken by it. Roughly 90% of the code lives in `BenKit`, a local Swift package — networking, auth, models, config — which is where to start reading.

Four rules the code follows

These are worth knowing before reading any screen, because every screen assumes them.

1. A refusal must never render as "nothing here." Every screen goes through `LoadStateView`, and loading / empty / refused / session-ended / rate-limited are visually distinct states. An empty list and a server saying no are different facts, and a UI that shows the same gray nothing for both teaches people to distrust it.

2. Website features map to native counterparts. Calendar to EventKit, locations to MapKit, uploads to the native camera and PhotosPicker, reports to PDFKit, sharing to the share sheet. If the website does it in HTML, the app does it the iOS way rather than wrapping a page.

3. One URL space, two front ends. `DeepLinkParser` maps the website's route table onto native screens, so `ishaunted://` and eventually `https://ishaunted.com/...` links land on the right screen.

4. The C# client patterns are the specification. `TokenSession` ports the website's bearer-token handler — single-flight refresh, expiry minus 30 seconds — and response mapping ports `WebApiClient` rule for rule: 401 means the session ended, 403 does not.

The feed

The app opens here. The feed is the front door: posts from groups and investigators, with media, hashtags, mentions and a group-verified badge.

Three lanes — **For You**, **Latest**, **Following**. "For You" is ranked by a category-match model whose weights are re-fit from labeled examples on the server, so it changes as the database grows rather than being a fixed rule.

Every image in this document is generated. Feed media here is simulated so the screenshots show a populated app; the frame captions say `SIMULATED`.



The feed — iPhone, dark mode

Notifications

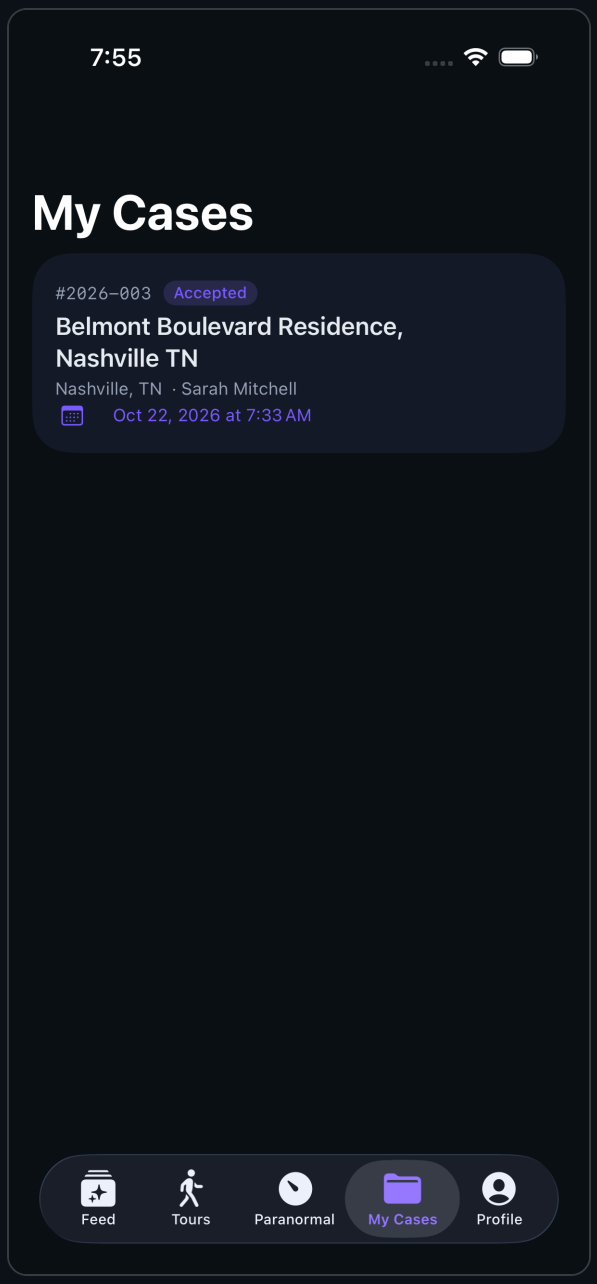
Bell in the top left. Notifications are server-side records rather than push-only, so the list is the same whether or not the person allowed push, and a device that was switched off misses nothing.



Notifications — iPhone, dark mode

Cases

A case is the unit of work: a client, a place, a history, and everything gathered about it. This list is what the signed-in person may see, which is not the same as everything that exists — the server decides, and the app shows what it is given.



Cases — iPhone, dark mode

Inside a case

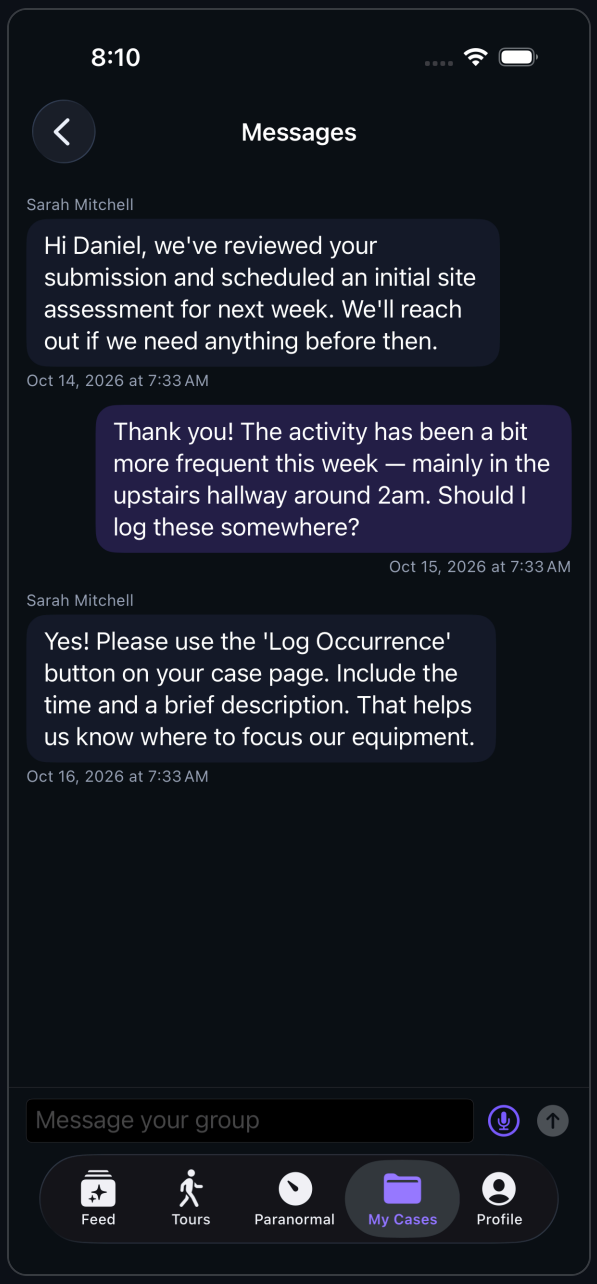
The case detail carries the original request, the timeline, files, investigations, and the people involved. Tabs are the same subjects the website uses, so somebody who knows one knows the other.



Inside a case — iPhone, dark mode

Case messages

Messages between the group and the client, in the case they belong to rather than in a separate inbox. A message that cannot be read in context is worth much less.



Case messages — iPhone, dark mode

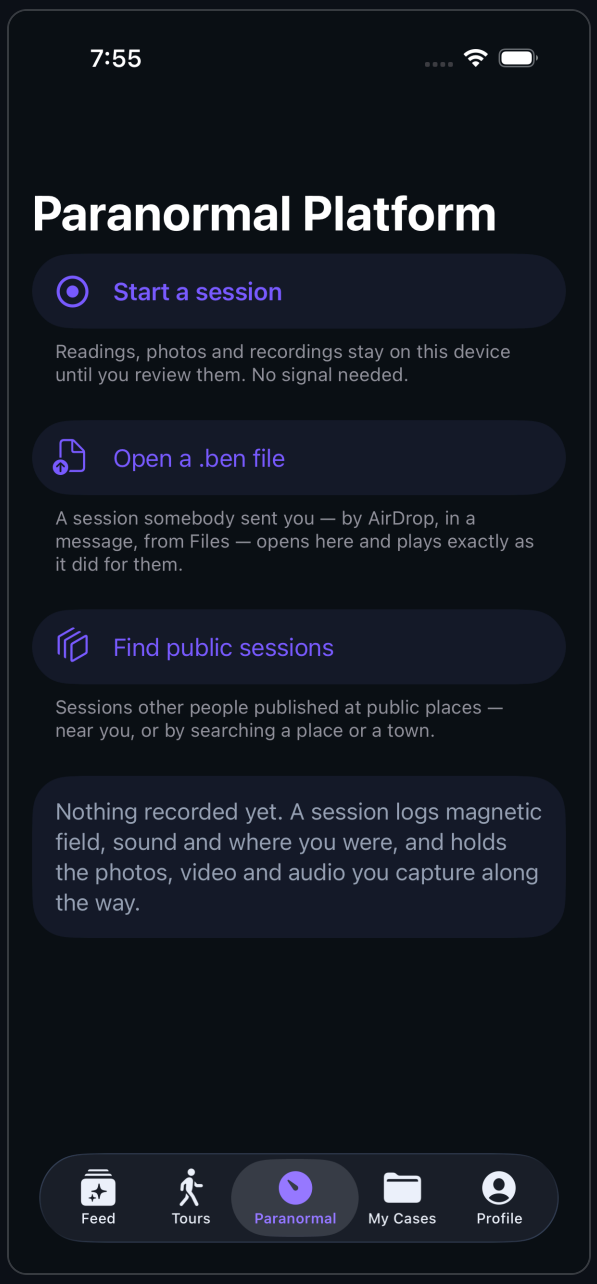
The Paranormal Platform — the reason the app exists

Everything above has a website equivalent. **This does not.** The Paranormal Platform turns the phone into an instrument, and it is the part a new developer should read first and most carefully.

The home screen lists past sessions and starts a new one. A session is a recording in the fullest sense: not a single measurement, but **every reading from every enabled sensor, for as long as it runs**.

Two more doors sit on this screen. **Open a .ben file** brings in a session somebody sent — a `.ben` is one sealed file holding the readings, the marks and the recordings — and it plays here exactly as it did for them; tapping

such a file anywhere on the phone does the same. **On the server, not on this phone** lists this account's sessions that were sent from another device or cleared from this one, and Download brings a whole night back. An imported session is the same three things a local recording leaves behind (the readings log, the rows, the media), so nothing downstream knows the difference; only a Source line on the review says where it came from. **Find public sessions** lists sessions other people published at public places — near the device, or at a place searched for by name or town — and downloads the archive's public copy, whose positions are all the place's public point rather than the path somebody walked.

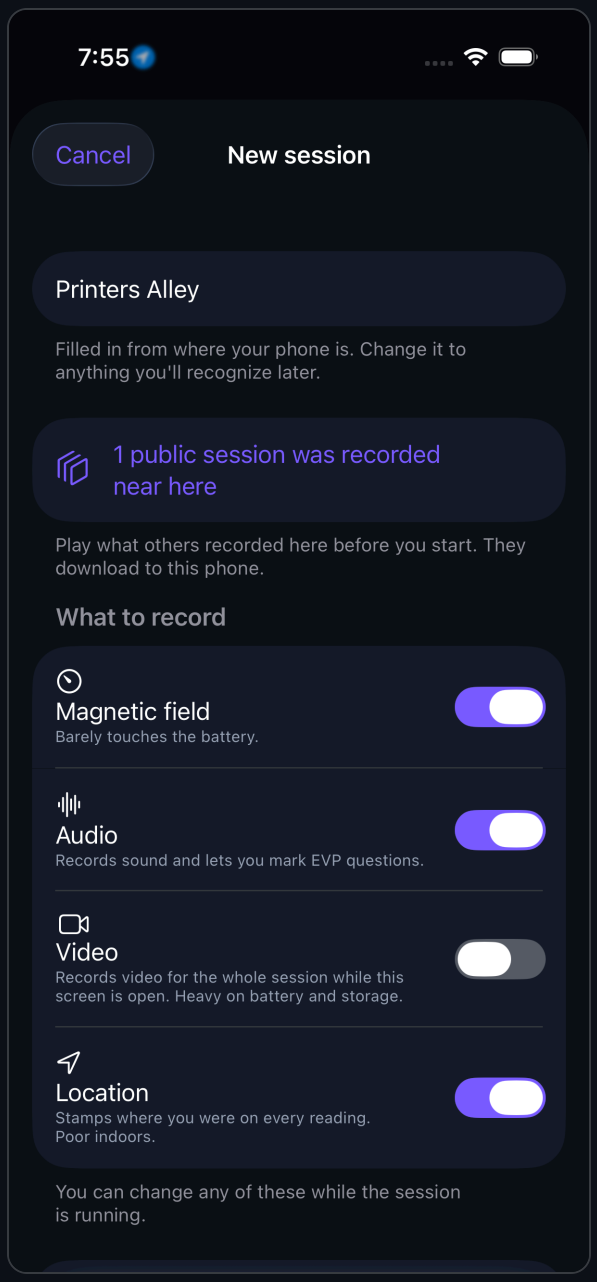


The Paranormal Platform — the reason the app exists — iPhone, dark mode

Naming a session

A session is named before it starts, not afterwards. The name is how it is found later, and asking at the end — when somebody is tired and packing up — is how recordings end up called "Session 4".

The sheet fills the name in from where the device is — one of IsHaunted's places within a quarter of a mile, or else Apple's street address — with nearby places offered as chips. It only asks to be typed when location is off. An investigation happening at this place is listed first and chosen already, and the sheet says when public sessions were recorded near here.



Naming a session — iPhone, dark mode

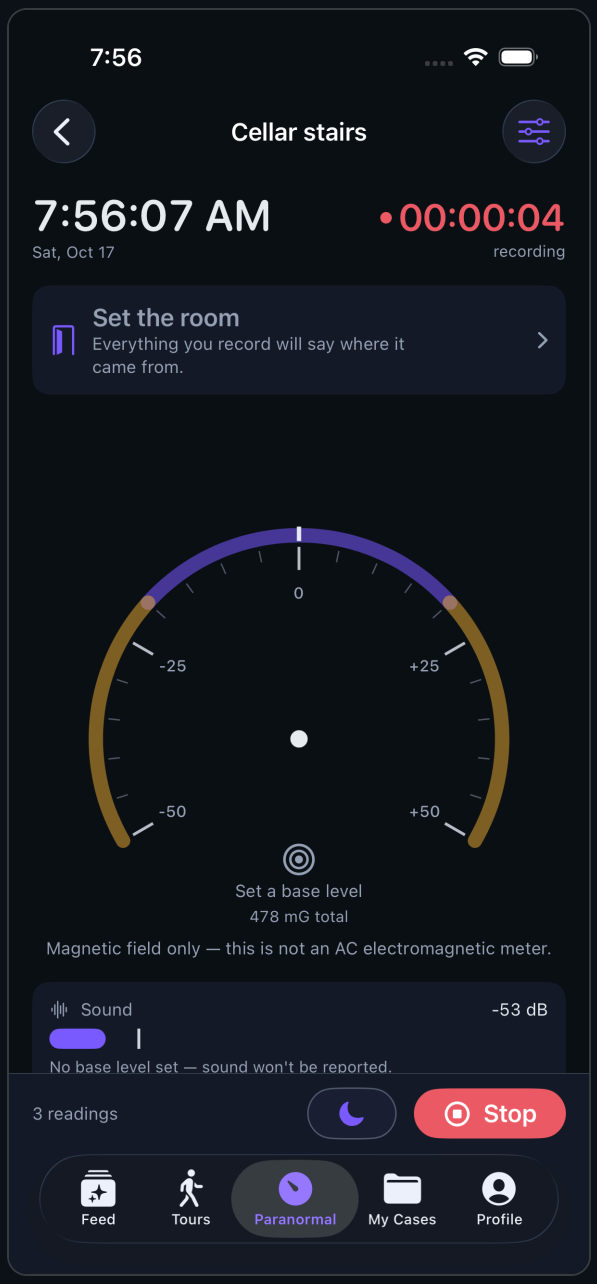
The live session

The instrument. The dial is a magnetometer readout; below it are sound, and whatever else is enabled.

> **The needle is missing on purpose until you set a base level.** It shows the **departure from base**, not the absolute field, because the absolute number means nothing on its own — the Earth alone reads about 500 mG and every building bends that. Before a base exists there is no departure to point at, so the dial says "Set a base level" instead of sweeping a scale nobody can interpret.

The needle is also damped rather than snapping to each sample. A real meter's movement has mass, and more usefully, a needle that jitters at 10 Hz is unreadable in the dark — which is the only place it is ever read.

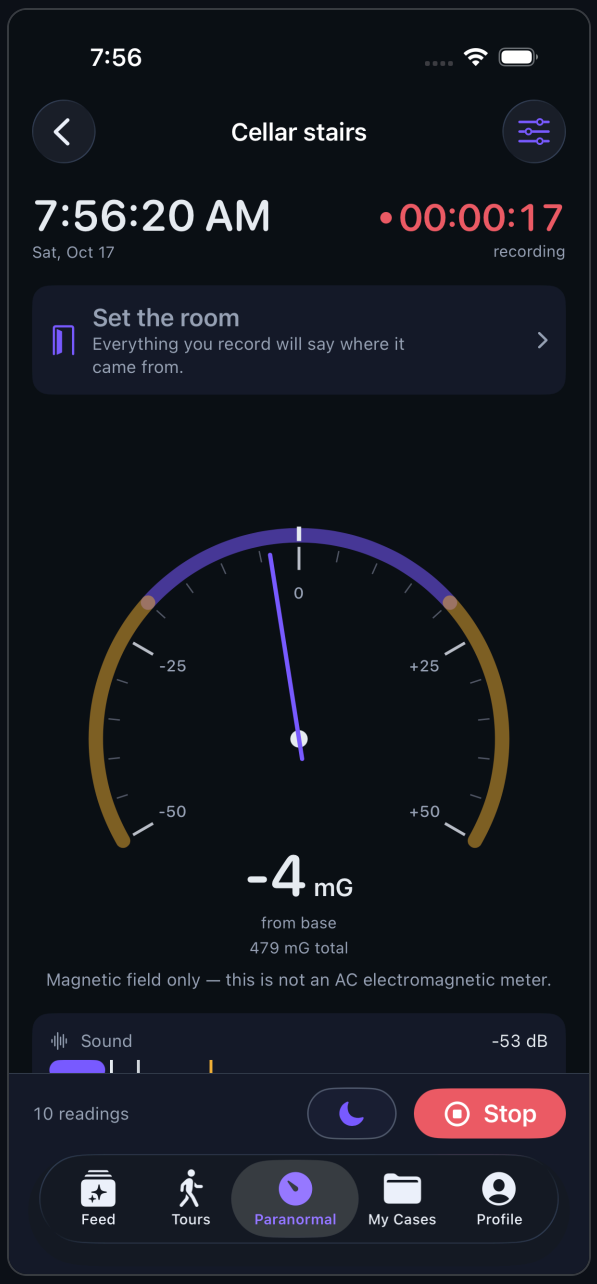
The caption under the dial is deliberate too: *"Magnetic field only — this is not an AC electromagnetic meter."* The app says what the instrument is not, because a person holding it in a basement cannot check.



The live session — iPhone, dark mode

With a base level set

Once a base is taken, the needle appears and the readout becomes a signed departure from it — `+12 mG from base` — with the report threshold marked on the dial. A needle pegged at the stop turns red rather than resting quietly at the maximum, because a reading at the limit is a reading whose real value is unknown.

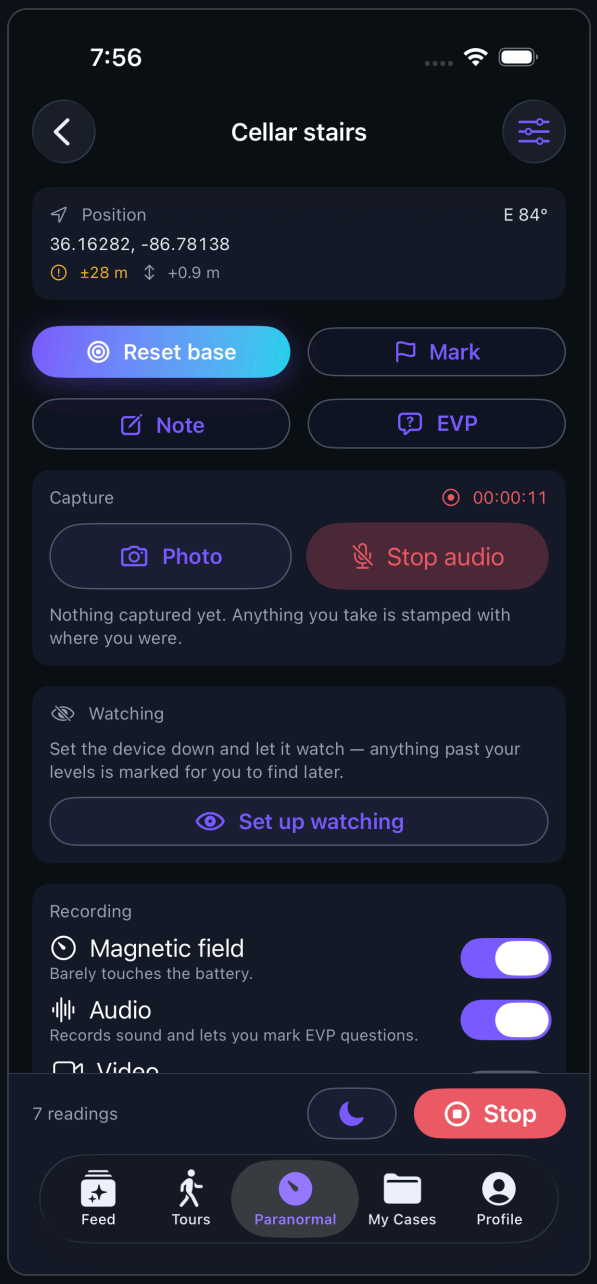


With a base level set — iPhone, dark mode

Everything the live session offers

Scrolling the live session reveals the rest of the instrument. Each control, and what it is for:

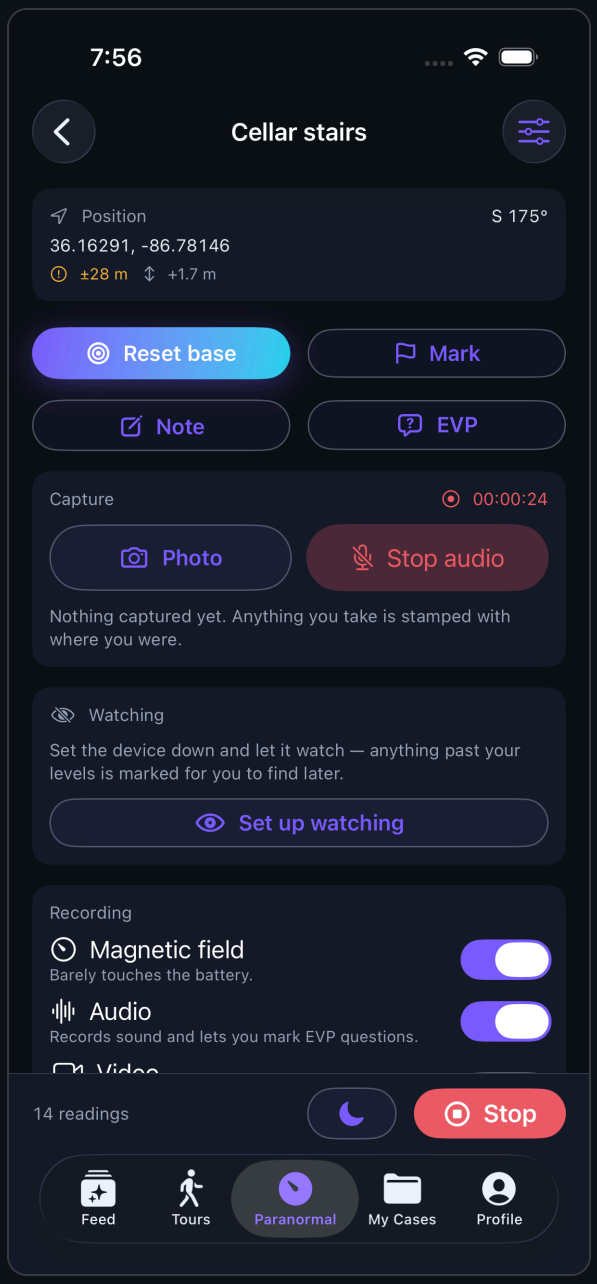
CONTROL	WHAT IT DOES	EXAMPLE
Position	Live GPS with its accuracy and altitude change. Accuracy is shown, not hidden, so a reading taken at ±28 m is not later mistaken for a precise one.	`36.16280, −86.78137 ±28 m`
Reset base	Takes the current field as the new normal. Used on entering a new room, because "normal" is a property of the room, not the building.	Move from the hall into the cellar, reset, and the dial answers the cellar's question.
Mark	Stamps this instant in the recording. Marking is cheap and reversible; it is the thing to press when something happens and there is no time to type.	A door moves — press Mark, describe it later from the replay.
Note	A typed or dictated note, timestamped and attached to the session.	"Three knocks, low on the wall, north side."
EVP	Ask a question and have the asking recorded as a marker, so the answer window is findable later instead of being hunted through an hour of audio.	"Is anyone here?" — the question is marked, the following silence is where to listen.
Photo	Capture into the session. Anything taken is stamped with where you were and which room you had named.	A photo of the stairs, filed to "Cellar stairs".
Video	With the Video channel on, the camera records for the whole session while the screen is open, with the session's own sound recording beside it.	Played back together, on one clock, in the review and on the website.
Watch for motion	While the phone is still, movement in the camera's view is marked "Motion detected" and photographed — the video keeps recording.	Propped on a stair: a shadow crosses the landing, and the moment is marked with a picture.
Audio	Continuous recording for the session, which is what EVP markers point into.	Runs for the whole session unless switched off.
Watching (sentry)	Put the device down and let it watch. Anything past your thresholds is marked automatically for you to find later.	Leave it on a landing during a break; come back to marks you did not have to be present for.
Room	Names where you are. Everything recorded afterwards says which room it came from.	"Cellar stairs" then "Boiler room".
Blackout	Kills the screen to near-black while continuing to record. A lit phone ruins both night vision and the video.	Tap to darken; tap again to return.
Sensor toggles	Each sensor can be switched off, with its battery cost stated plainly — "Barely touches the battery" against video's heavier cost.	Long overnight sit: magnetic field and audio on, video off.



Everything the live session offers — iPhone, dark mode

Marking, and notes

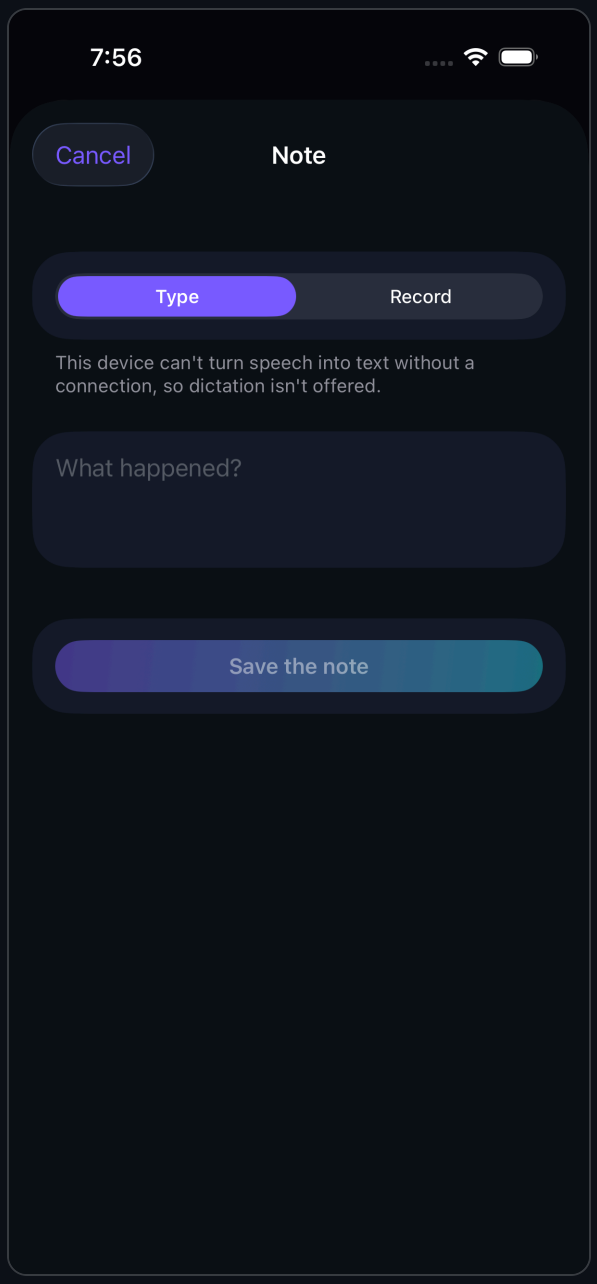
A mark is a moment; a note is a moment with words attached. Both land in the same timeline as the sensor readings, which is what makes the replay coherent — the readings, the marks, the notes, the photographs and the audio are one record, not five.



Marking, and notes — iPhone, dark mode

Writing a note

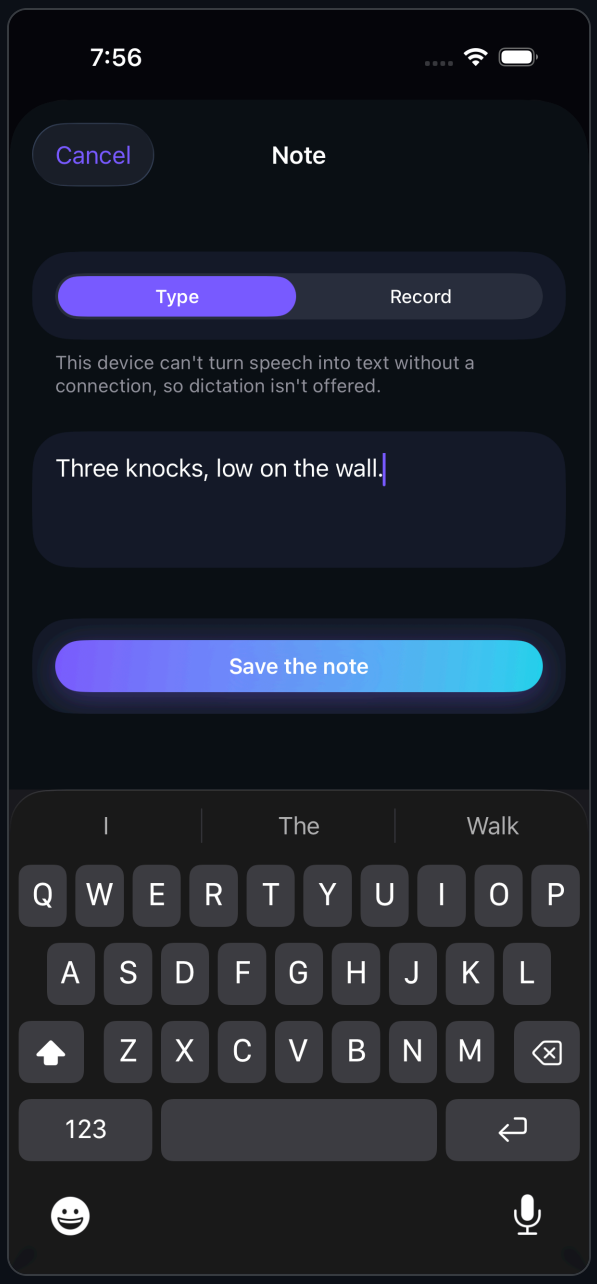
Notes can be dictated, because typing in the dark with gloves on is not realistic. The note is stamped with the time and the current room.



Writing a note — iPhone, dark mode

A note as it is typed

Typing is the fallback where the device cannot transcribe offline, and the correction path where it can — a transcription is a machine's best guess, and the person who spoke is the authority on what they said. Saving drops the note onto the session timeline at the current moment.

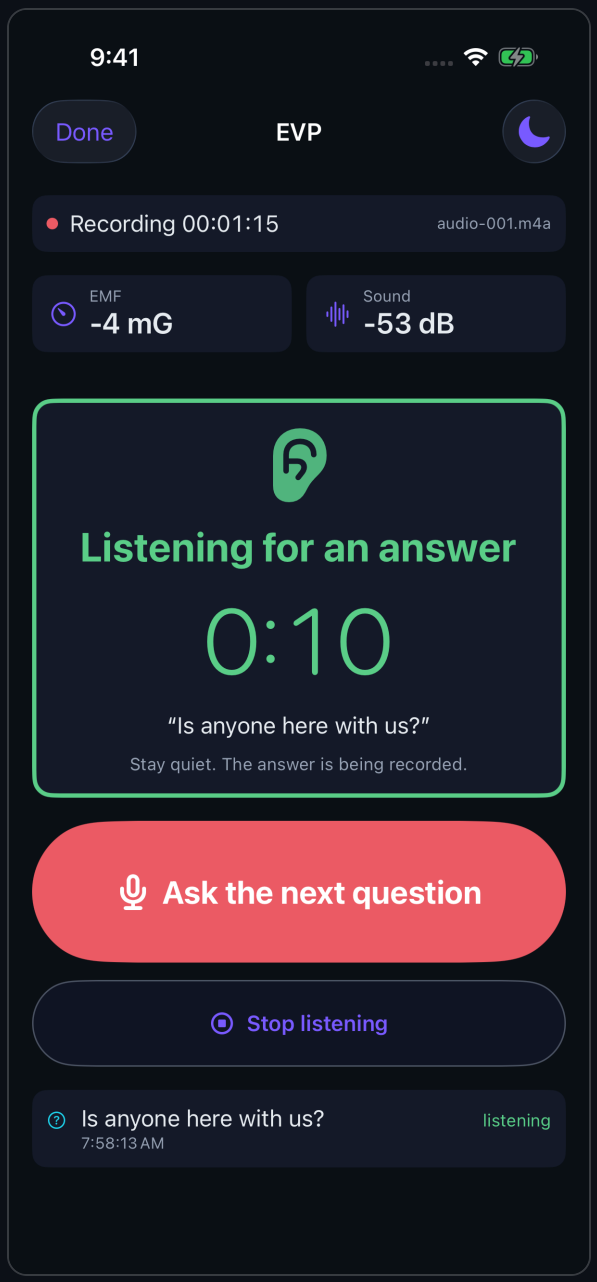


A note as it is typed — iPhone, dark mode

EVP

Question, silence, question — the shape of an EVP session, and this screen is built for doing it in the dark while talking to a room. One large button takes each step and says what pressing it will do: **Ask a question**, then **Done asking — listening**, then **Ask the next question**. The screen is red while somebody asks, with the question as it is written down, and green while the room has the floor, with how long it has had to answer. The magnetic field and the sound stay on screen throughout, so nobody has to leave the wait to see whether something changed. The pictured screen is listening (version 1.1.5).

The wait is what is being recorded, not the question. Marking only the moment somebody spoke would leave a reviewer hunting through hours of tape; bracketing the silence after each question turns the same recording into a list of places to listen. The questions and their brackets go onto the session timeline, so the replay puts them back beside the readings taken at the same moment.



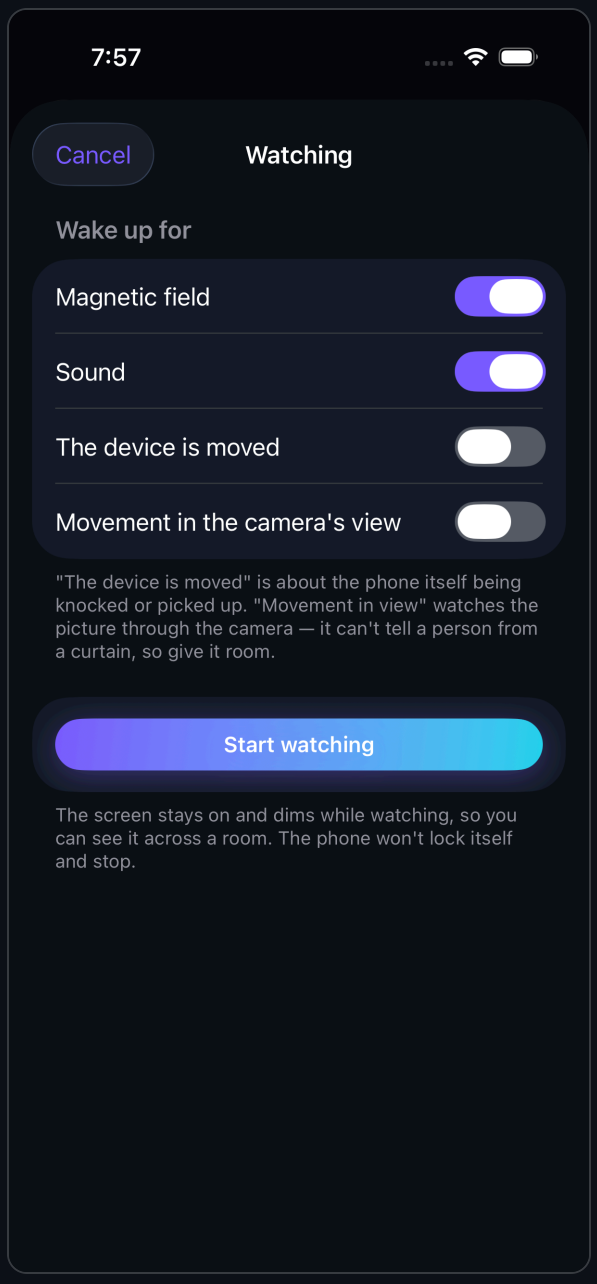
EVP — iPhone, dark mode

The sentry

The sentry watches while nobody is looking at the phone and marks the timeline by itself. Four triggers, each switched on separately: magnetic field and sound, measured as a departure from the base level; the device being moved, which is what matters when a tripod is disturbed; and movement in the camera's view. What it

does when one fires is mark the timeline. Recording video is the Video channel's job, for the whole session; **Watch for motion** is separate from arming and works whenever the phone is still, marking and photographing what moved (2026-09-27).

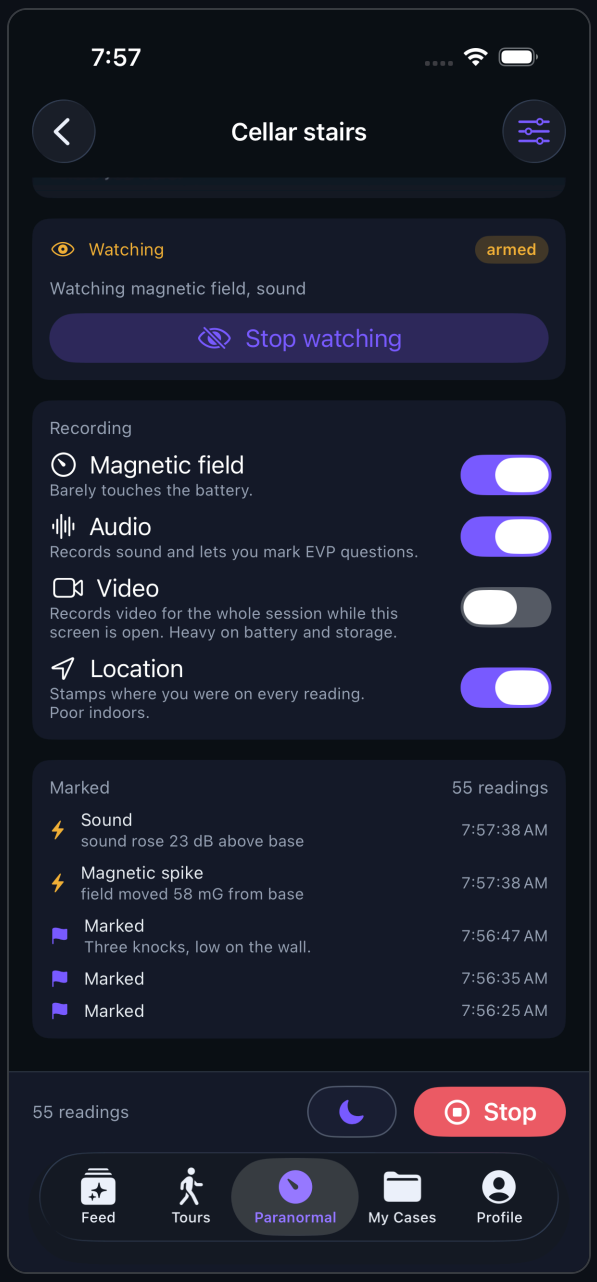
Arming is refused, in words, when it would be pointless — a magnetic threshold with no base level to measure against is not a threshold, and scene motion needs the camera switched on.



The sentry — iPhone, dark mode

Armed

Every mark the sentry makes carries its own kind — `sentry_emf`, `sentry_sound`, `device_moved`, `scene_motion` — so a machine's notice is never mistaken for a person's on replay. Who noticed a thing is part of the evidence.



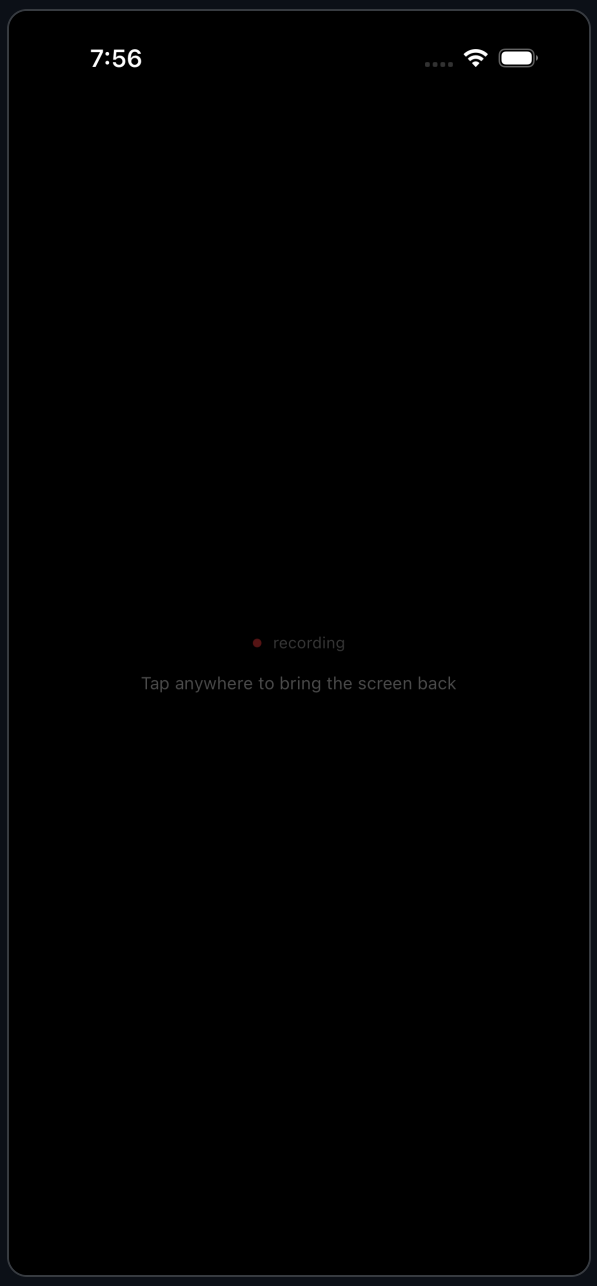
Armed — iPhone, dark mode

Blackout

The screen off, while everything underneath keeps running. A lit phone in a dark room reaches the recording, the room and everyone in it, so this takes the brightness to nothing and covers the screen in black while the

session keeps logging.

It is deliberately not a lock: a single tap anywhere brings it back, because fumbling for a specific control in the dark is exactly what somebody cannot do.



Blackout — iPhone, dark mode

How a recording is replayed — and why

> **The Paranormal Platform records every reading, continuously, for the whole session — and selecting a past session plays the entire recording back as it happened. This is deliberate and it is the central idea**

> **of the feature.**

It does not show you one reading at a time, and it does not ask you to pick a measurement out of a list. It replays the session: the dial moves as it moved, the marks arrive when they arrived, the notes appear when they were written, and the audio and video run alongside them on the same clock.

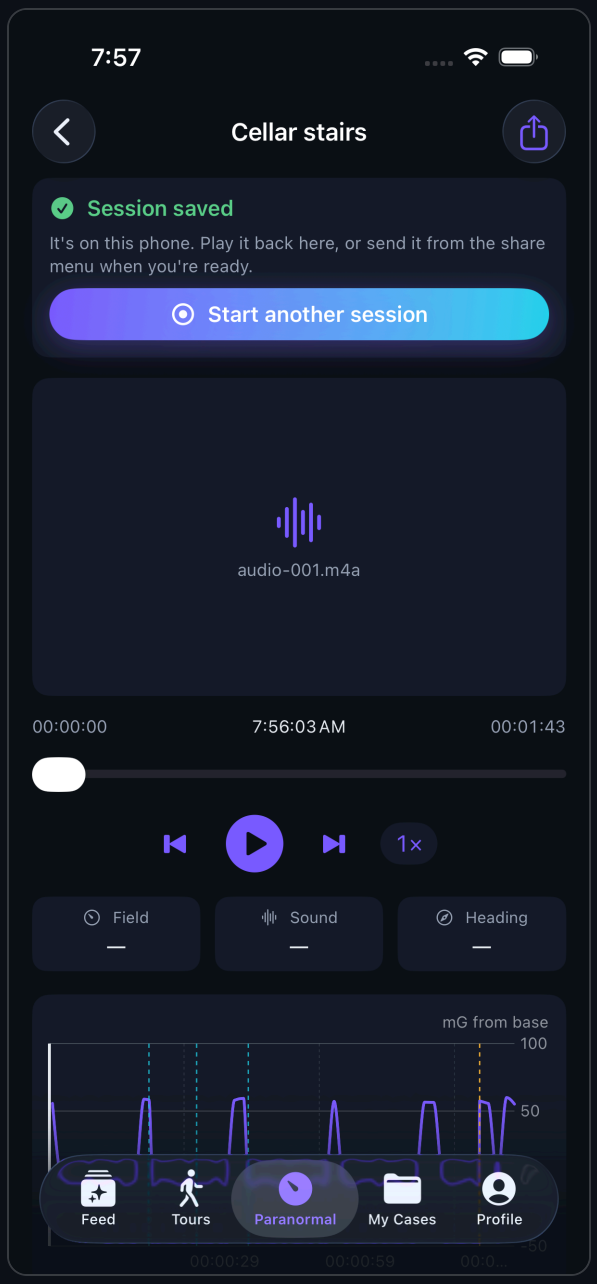
The reason is that a single reading proves nothing. "47 mG at 02:14" is an anecdote. What matters is what the field was doing for the ten minutes around it, whether anybody was moving, what the room was, and what was heard at the same moment — and the only way to judge that is to watch it happen again.

The whole recording can then be submitted as evidence, not just a clip or a screenshot. It attaches to a case or is published to a place's archive, and it plays back in a player in the web app — the same replay, in the browser, for people who were not there. That is what turns a night's work into something a client, a colleague or a stranger can actually assess.

Ending a session

Stopping opens the review: what was captured, how long it ran, what was marked. It says the session is saved on the phone and offers **Start another session**, with the same channels. The video plays with the session's sound; photographs sit in a strip under the player, and each glows and grows a little as playback passes the moment it was taken — tap one to pause and see it full size, tap again to carry on. From here the session can be sent to the server or exported — both as one sealed `.ben` file — attached to a case, or published to a public place's archive.

Pressing **Start a session** while one is still open saves a recording one (or replaces one that never started) after asking, instead of making somebody deal with it first. The New session sheet fills in where you are — one of IsHaunted's places nearby, or else the street address — puts an investigation happening here first, and says when other people published sessions nearby. If a permission the app needs was refused, the switch says so and offers Settings.



Ending a session — iPhone, dark mode

A group session, started by the lead

Item 252. A tour's guide, an event's organizer or door staff, or an investigation's lead starts the Paranormal Platform for everybody at once. The server (`FieldLaunchService`) finds who is registered, writes a card in the feed — public when the thing is, otherwise a `FieldLaunchNotice` only its people can read — and pushes to every registered phone through APNs. Nobody is forced in: the push, the card and the Paranormal Platform's **Happening now** all open the same link, `ishaunted://field-kit/launch/{id}`, and `JoinLaunchView` turns it into a session already set to the thing and its place (`startSession(joining:)`), pending until Start.

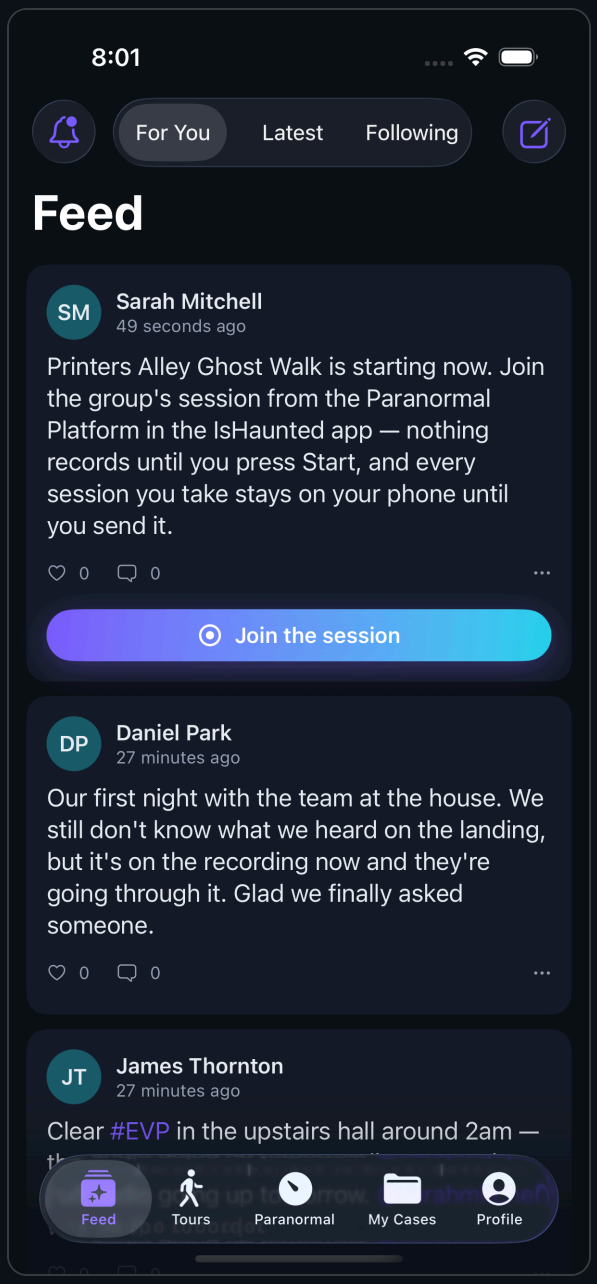
Push is `PushRegistrar` with an `AppDelegate` adaptor: permission is asked at moments it plainly helps (a reserved seat, a booking, something coming up), the token is sent to `api/me/push-devices` while somebody is signed in, and removed in `SessionStore.beforeSignOut`. Taps finish on the main thread — the `async` form of `didReceive` crashed the app.



A group session, started by the lead — iPhone, dark mode

The launch in the feed

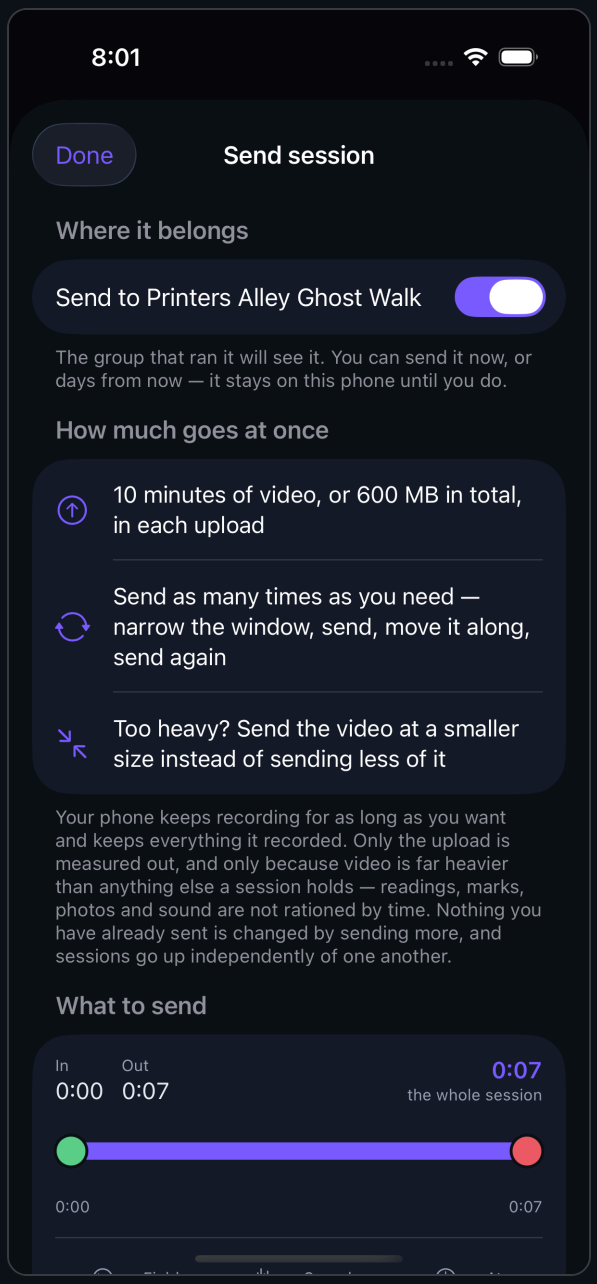
`FeedPostRecord.launch` carries the card's launch; `FeedCardView` draws **Join the session** outside the card's combined accessibility element, so it is a real button to VoiceOver and to UI tests. The card leaves the feed six hours after the thing ends.



The launch in the feed — iPhone, dark mode

Sending it to the night

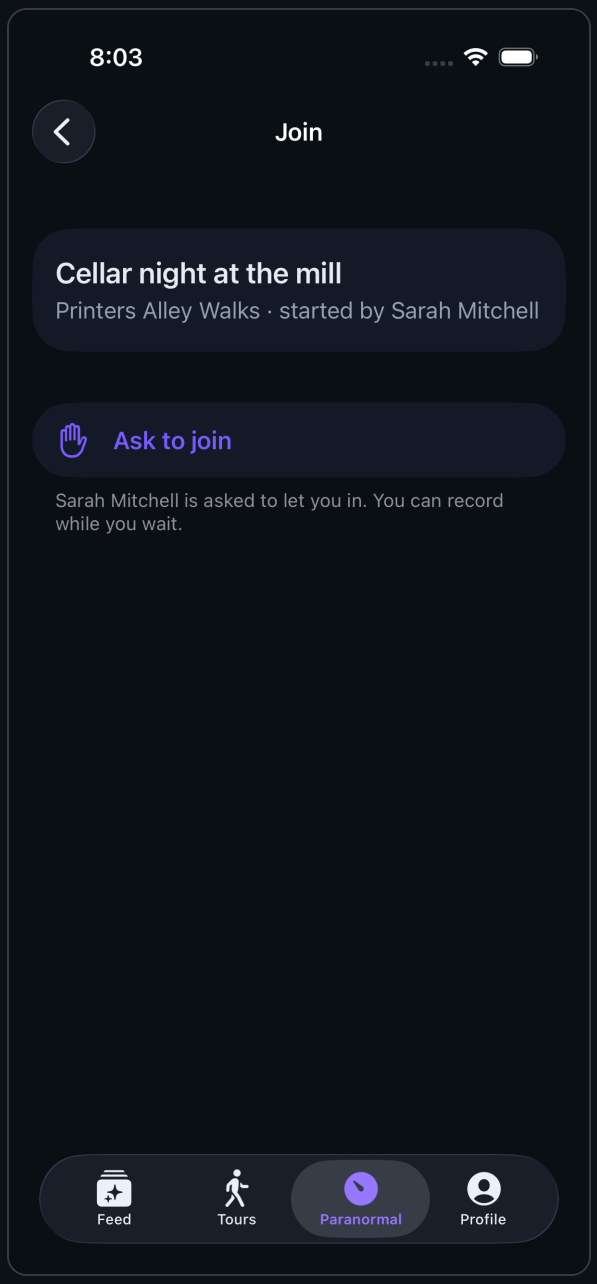
A joined session remembers its launch (`fieldLaunchId`) and the tour date or event. The send screen replaces the investigation picker with **Send to …**; the upload sends the launch and the server resolves where it goes — so it works days later, and for a walk-up let in after they recorded. Each part of a long session goes under its own id (`SessionWindow.uploadId`), so the next ten minutes no longer replaces the first. Rows in the Paranormal Platform say Sent or Not sent.



Sending it to the night — iPhone, dark mode

Joining by the lead's code

The lead's QR carries a secret join token (`/field-kit/join/{token}`, claimed as a universal link). `JoinByCodeView` asks the server where the scanner stands: registered or public — straight in; otherwise sign in and **Ask to join**, record while waiting, and the view looks again every few seconds so a yes moves them in. A home or a private client case has no token.



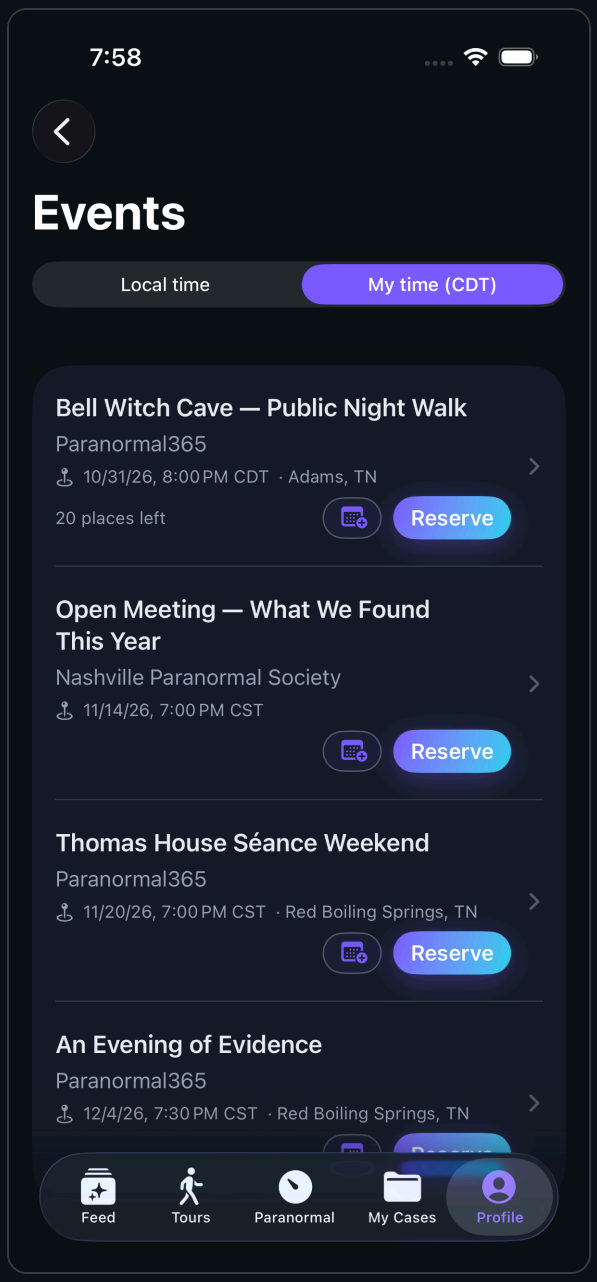
Joining by the lead's code — iPhone, dark mode

The lead's page

`LaunchForGroupView` lists what the signed-in person may launch now (three hours before until the end); `LaunchDetailView` shows who it reached, the QR code, and anybody asking, with Let in and No. On an iPad the split view's detail column gives each section's `NavigationStack` its own identity (`.id(section)`) — without it, a link into a page below a section's front screen, such as the lead's "wants to join" notification, stopped at the front screen.

Events

Public events — tours, open investigations, group meetings. Someone who attended can offer what they captured; a group member reviews it, and an accepted submission becomes part of that event's public record, credited to them.

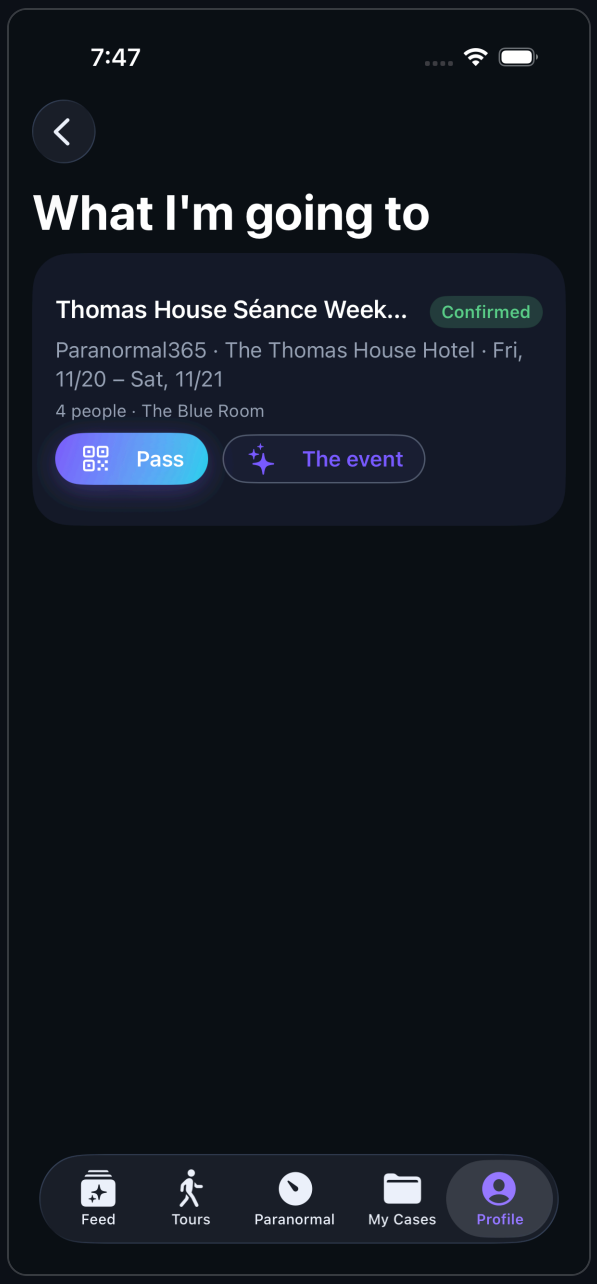


Events — iPhone, dark mode

Hosted events: what I'm going to

Hosted events are weekends and evenings a group or venue runs with rooms or seats (item 235). A guest's bookings are listed under **What I'm going to** in Profile, one row per event: the live booking, or the most recent

one. **Pass** and **The event** are buttons that push onto the router; two `NavigationLink`s in one `List` row both answer to the last one.



Hosted events: what I'm going to — iPhone, dark mode

A pass that opens with no signal

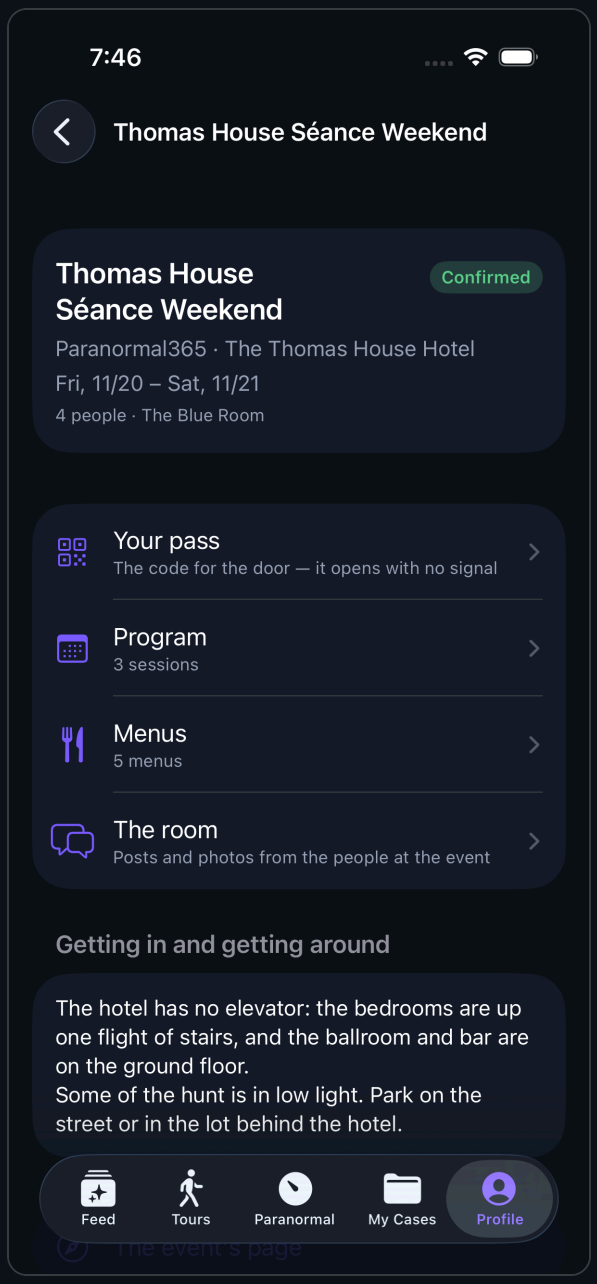
The QR code is drawn on the phone from the pass token with `CIFilter.qrCodeGenerator()` and saved by `PassCache`. A cellar with no signal still opens it, and a withdrawn pass is shown as withdrawn, never as a blank. The six-character code under it is what the door types when a camera will not read the screen.



A pass that opens with no signal — iPhone, dark mode

The event's own screen

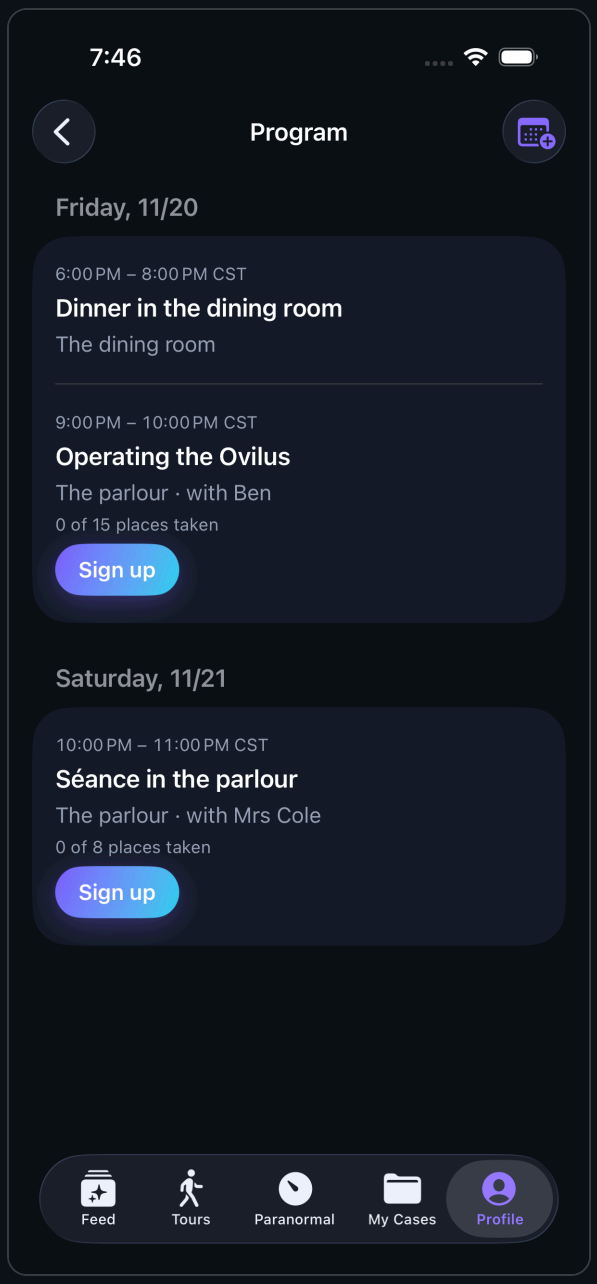
`EventHubView` gathers the pass, the program, the menus, downloads, the room and the venue's access notes. A row appears only when there is something behind it: a 403 or 404 from its endpoint is read as "not for this guest" (`absentIsNil`), not as an error.



The event's own screen — iPhone, dark mode

The program

Sessions night by night on the venue's clock, with sign-up, a waiting list, and a calendar export. A change the guest has not seen yet is badged on the hub.



The program — iPhone, dark mode

Menus

Each published menu drawn in its own design (`DesignedMenuView`): the design's paper and inks, frame, ornament and typefaces, with the same words the website and the PDF print (`EventMenuWording`, ported from the server's `EventMenuPresentation`). The designs and the list of font files come from `GET api/public/menu-designs`; `MenuFontLibrary` fetches only the files a design uses from `api/public/menu-fonts/{file}`, keeps them in Caches and registers them for the process, so a design added on the server reaches the app without an update. Until a face arrives the menu is drawn in the system face of the same kind.

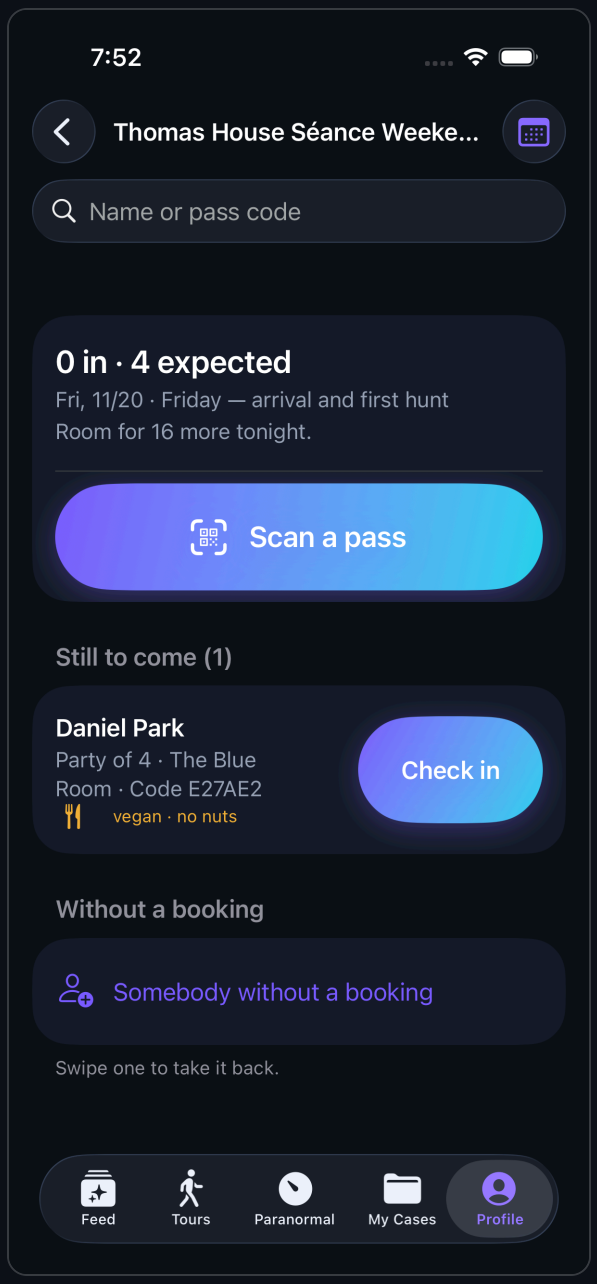
PDF opens the server's own PDF in Quick Look. Published menus are public, so the calls are anonymous and the event's page links to them for anyone.



Menus — iPhone, dark mode

Running the door

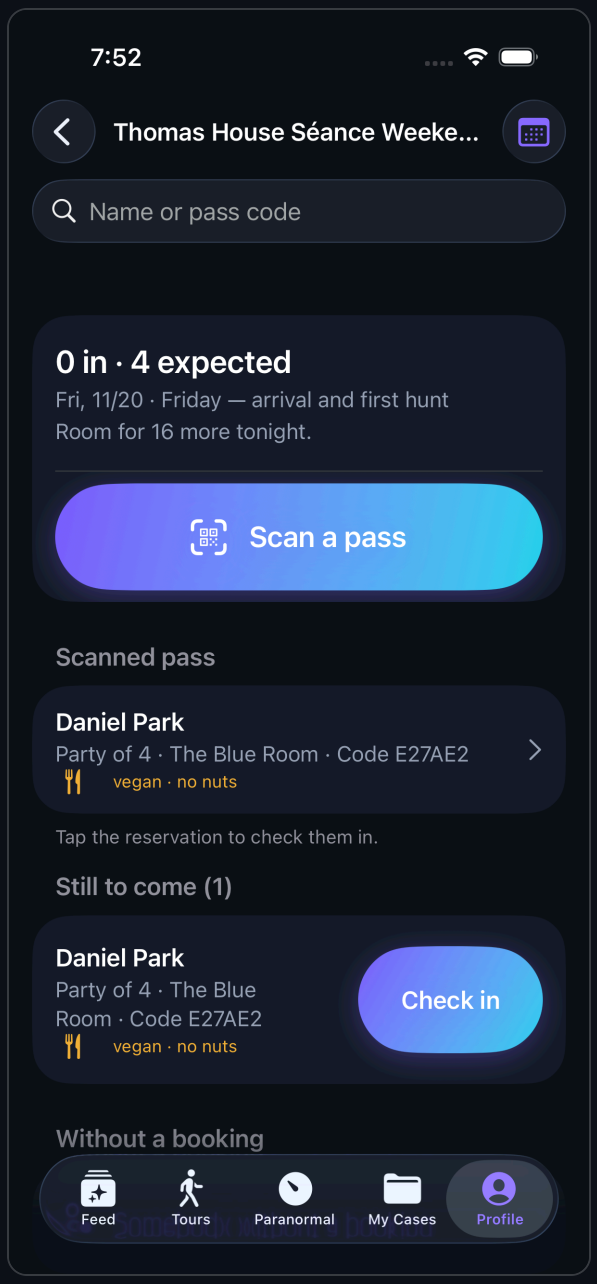
Doors I'm running appears in Profile only for somebody who may run one: `GET api/me/hosted-event-duties` covers members, accepted helpers and venue staff. `DoorStore` keeps the night's list in `DoorCache`, so the list, the search and check-in keep working with no signal. Arrivals made offline are queued in `waiting.json` and sent in order with the time they happened; the server's `DoorClock` keeps the earlier of two arrivals.



Running the door — iPhone, dark mode

A scanned pass

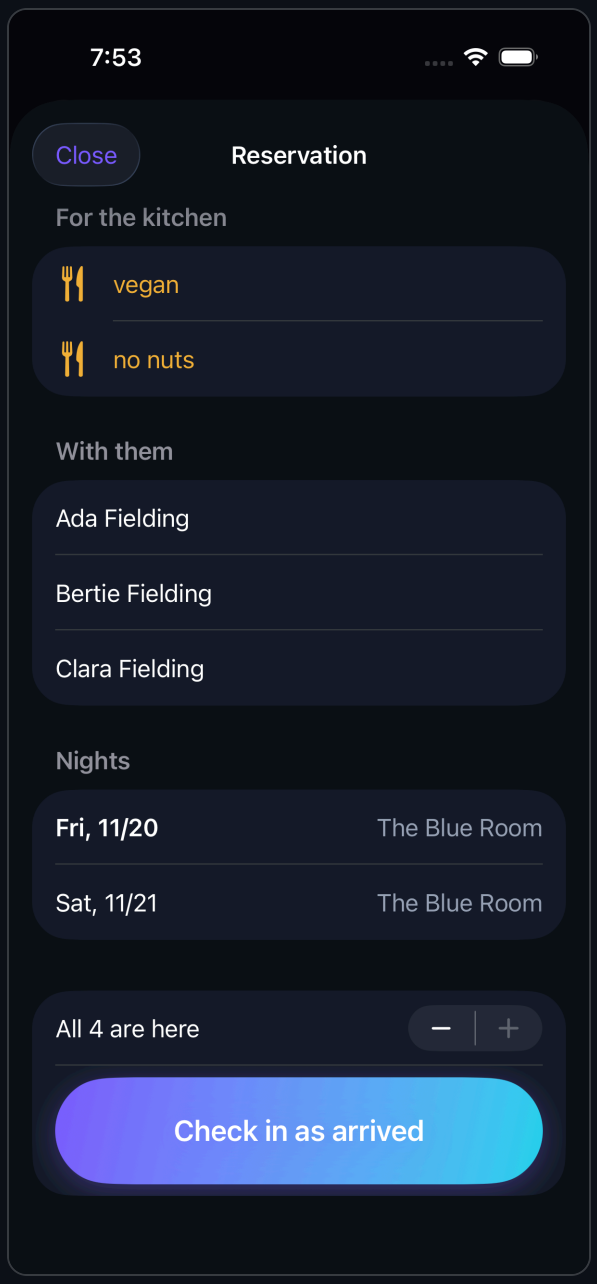
The scanner screen is only the camera (`DataScannerViewController`). A read code is looked up and shown as the reservation, and nothing is checked in until the person on the door taps it. On a simulator, `-doorScanCode <token>` stands in for the camera.



A scanned pass — iPhone, dark mode

Checking a party in

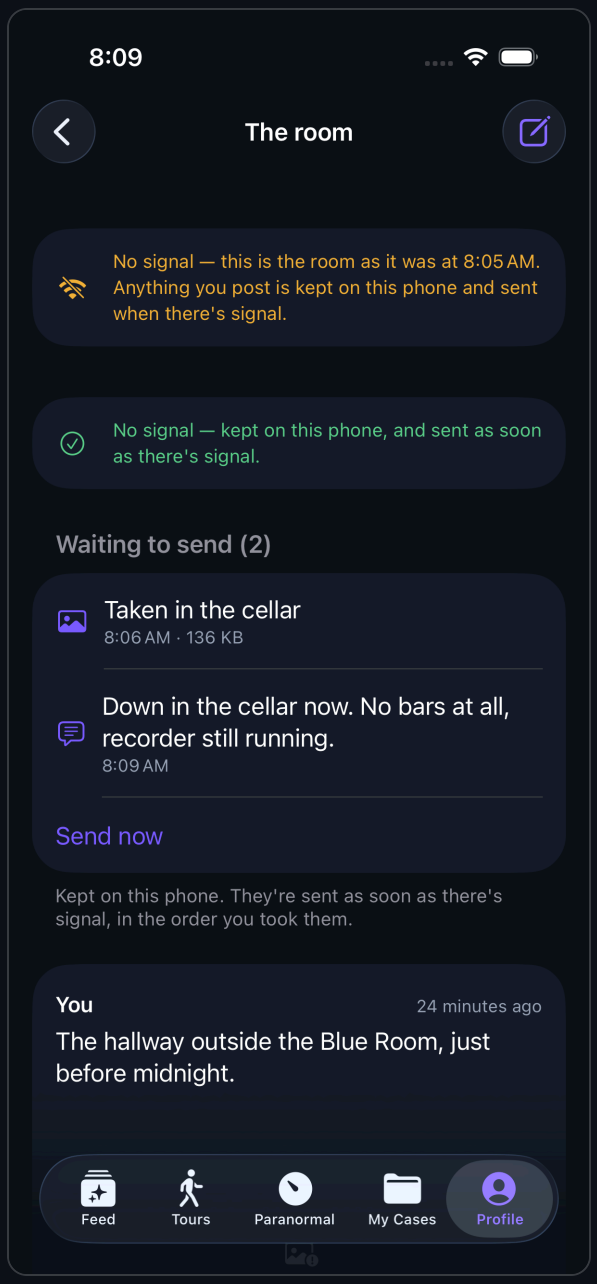
The reservation shows who, how many and where on each night, and adjusts the head count before **Check in as arrived**.



Checking a party in — iPhone, dark mode

Photos with no signal

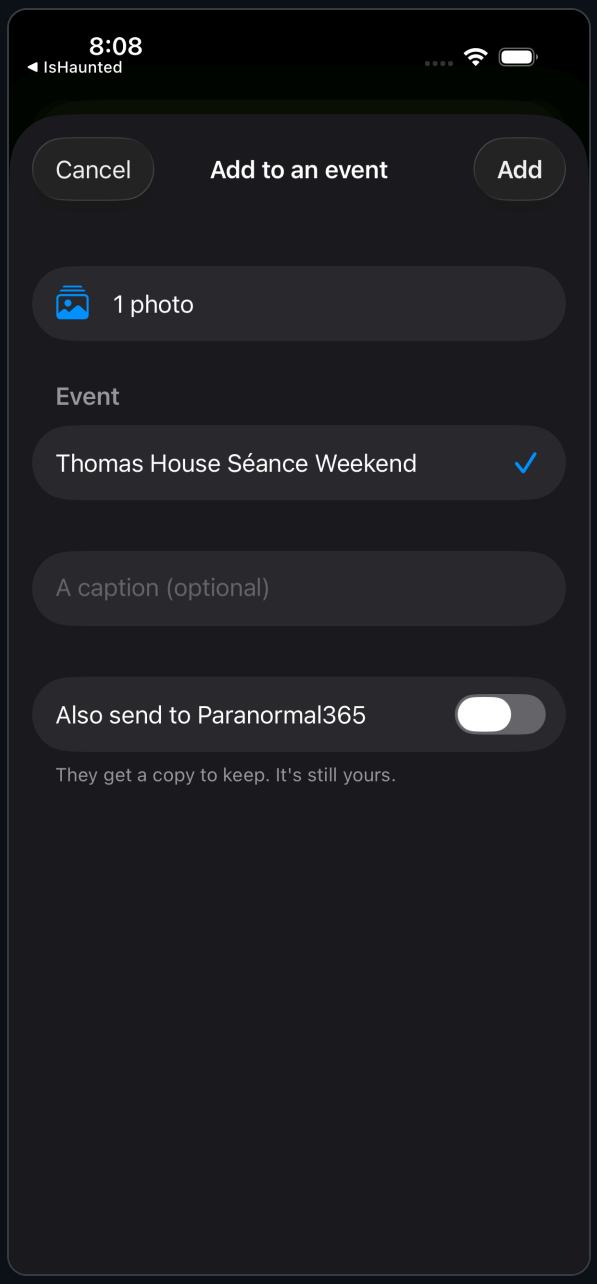
A post to the event's room made with no signal is kept in `RoomOutbox`, in the App Group container `group.com.ishaunted.ios`, and sent oldest first by `RoomOutboxSender` when the network returns or the app comes forward. A post the server refuses is kept, with the server's sentence, for the guest to remove.



Photos with no signal — iPhone, dark mode

Sharing from Photos

The `IsHauntedShare` extension offers the events the app last wrote to `ShareableEvents`. It has no sign-in of its own, so it only adds to the outbox, and the app sends.

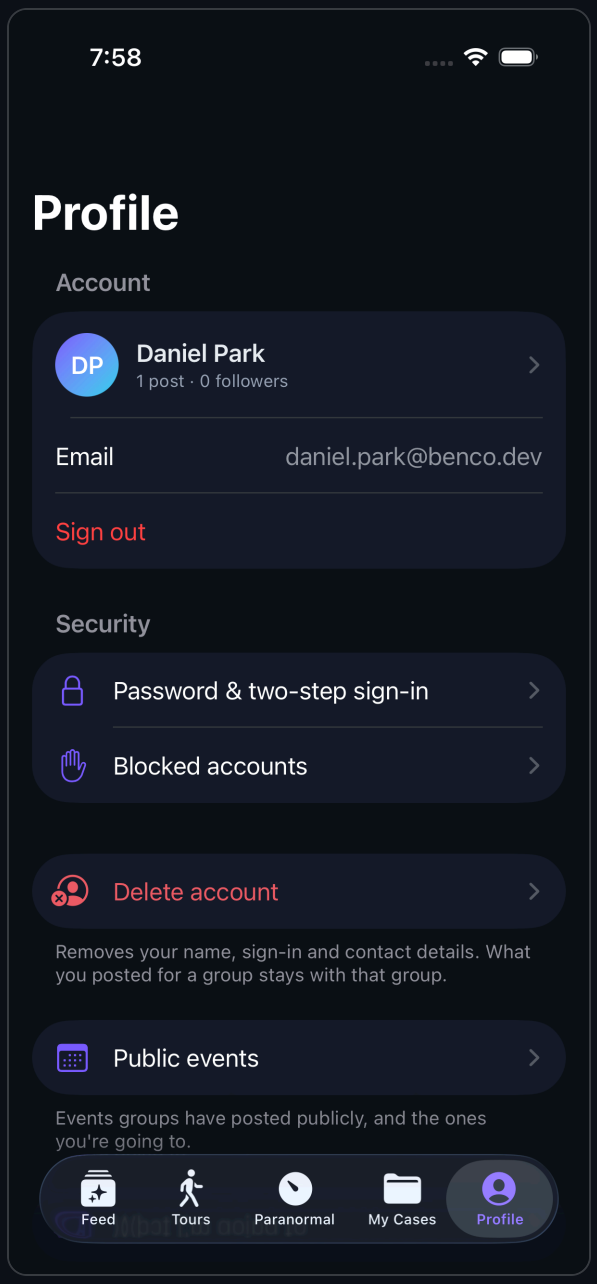


Sharing from Photos — iPhone, dark mode

Profile and account

Account, security, environment, and the About and Privacy screens Apple requires to be reachable without an account.

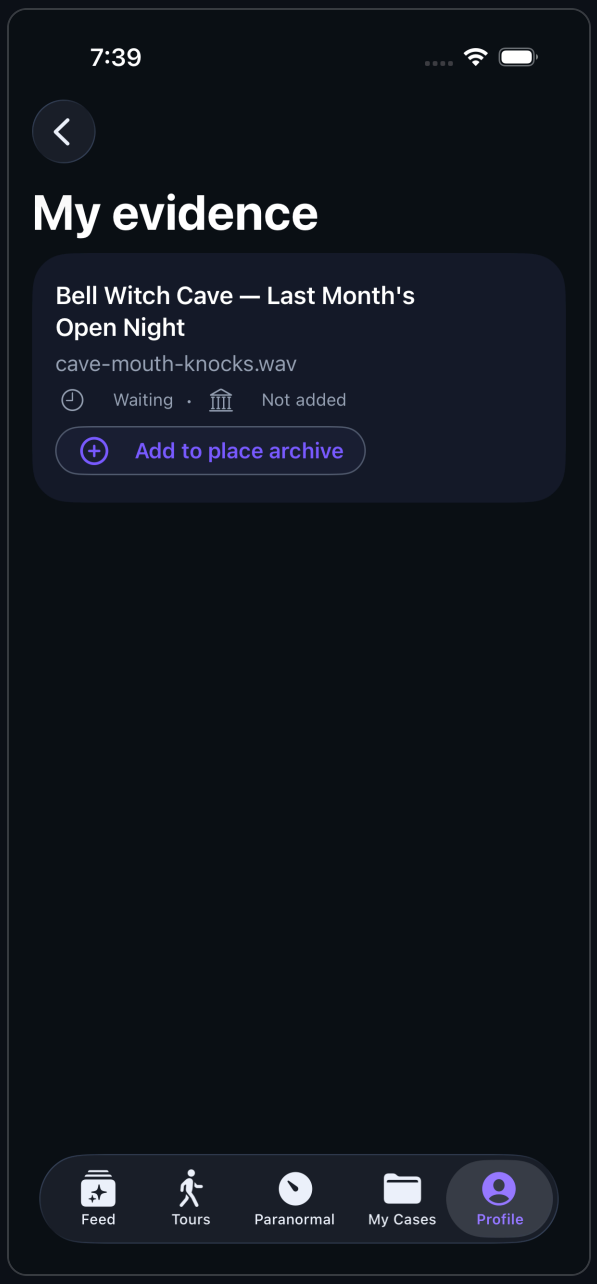
The app deliberately shows somebody only what applies to them. A person investigating alone does not see group administration; someone who is not on a ghost walk does not see its controls.



Profile and account — iPhone, dark mode

My evidence

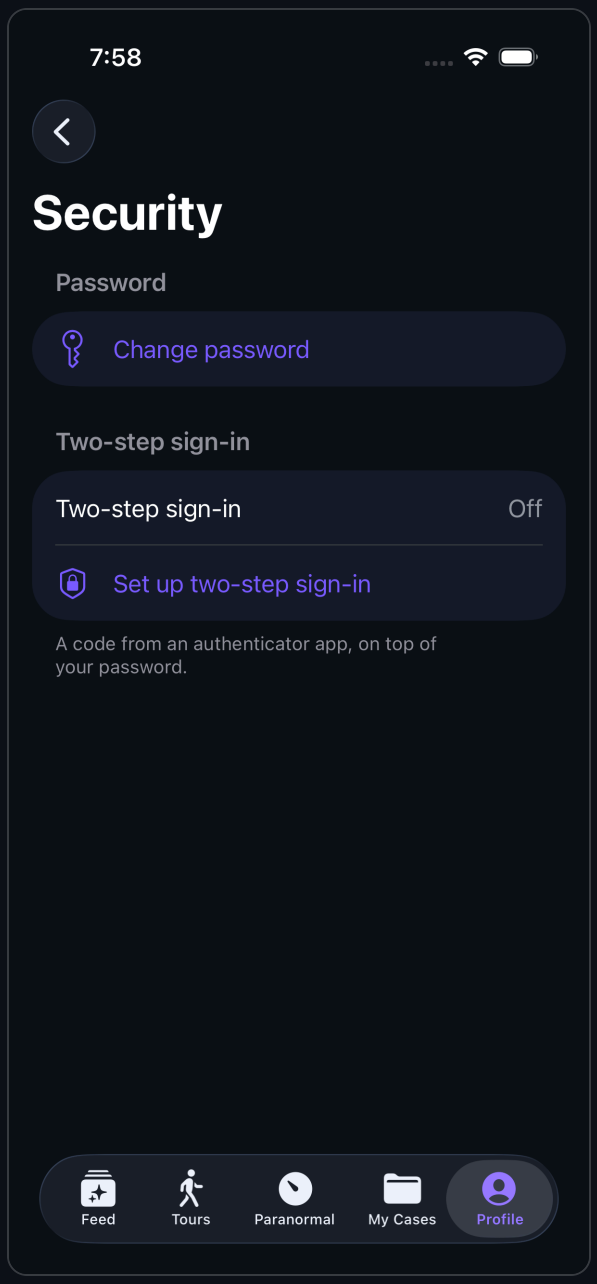
What this person has captured and offered, and what became of it. A guest who photographed something at somebody else's event keeps their own copy and can see whether it was accepted.



My evidence — iPhone, dark mode

Security

Password, two-factor, sign-out, and account deletion. Deletion anonymises rather than shredding where a record must survive — and the server refuses to delete a group's last owner, because an ownerless group is worse than a deleted account.



Security — iPhone, dark mode

Building and running it

From `Ben.iOS/`:

```
./scripts/test.sh                          BenKit unit tests, host only, no simulator
./scripts/build.sh                         unsigned simulator build (CI compile check)
./scripts/run-sim.sh                       build, install and launch on iPhone 17 Pro
./scripts/run-sim.sh "iPad Pro 13-inch (M5)"
OPEN_LINK="https://ishaunted.com/events" ./scripts/run-sim.sh    deep-link on launch
```

Run the API first — `dotnet run` in `Ben.Data.WebApi` on :5252 — and pick **Dev** under Profile → API environment. The simulator reaches the Mac's localhost directly.

> **One trap worth knowing.** A fully unsigned build (`CODE_SIGNING_ALLOWED=NO`) cannot use the simulator Keychain, so token persistence silently fails and every launch looks signed-out. Only `build.sh` disables signing; `run-sim.sh` and Xcode use ad-hoc signing on purpose.

Screenshots for this document are captured by `IsHauntedUITests/DeveloperDocCaptureTests`, which needs its flag passed as a shell environment variable — `TEST_RUNNER_BEN_DOC_SHOTS=1` before `xcodebuild`, not as a build setting, or the test silently skips.